

dataQ 승인 프로세스: 다형 참조와 설계 반전

dataQ에서 "승인"이라는 말은 단어·용어·도메인·코드 네 종류를 모두 묶는 개념이다. 이 네 가지는 데이터 모델상 거의 공통점이 없다. 단어는 영문약어를 가지고, 용어는 단어들의 조합을 가지고, 도메인은 타입과 길이를 가지고, 코드는 값 집합을 가진다. 그런데 운영 측면에서는 하나의 흐름이다. 누가 등록 요청을 하고, 누가 검토하고, 누가 승인·반려하고, 반려되면 사유가 남고, 반려되면 다시 고쳐서 올릴 수 있어야 한다.

처음엔 네 종류 각각에 승인 테이블을 따로 두는 선택지도 있었다. 단어 승인·용어 승인·도메인 승인·코드 승인. 테이블 네 개, 쿼리 네 벌, 컨트롤러 네 벌. 구현은 가장 단순하지만, 승인 화면 하나에서 네 종류를 섞어 보여주고 싶을 때마다 UNION을 쓰는 구조가 된다. 네 종류에 나중에 다섯 번째(예: 도메인 그룹)가 붙으면 테이블 다섯 개, UNION 다섯 갈래가 된다.

그래서 `TB_APPROVE_STAT` 하나에 `OBJ_TYPE` (WORD/TERM/DOMAIN/CODE)과 `OBJ_ID`를 함께 담는 구조로 갔다. 말 그대로 다형 참조다. 조회는 `OBJ_TYPE`으로 필터하고, 조인은 상황에 따라 동적으로 건다. 대신 DB 레벨에서 FK를 걸 수 없다는 약점을 안고 간다. `OBJ_ID` 하나가 네 테이블 중 어느 것을 가리키는지가 타입 컬럼에만 의존하기 때문에, 타입이 망가지면 조인 자체가 이상한 방향으로 간다. 이 리스크는 "컨트롤러 레이어에서 타입을 반드시 명시적으로 세팅한다"는 규칙으로 덮었다. 약점을 알고 받아들인 거다.

처음 설계의 엄격함, 그리고 그걸 뒤집은 지점

첫 설계는 단어 승인과 용어 등록을 엄격하게 묶었다. 미승인 단어로는 용어를 등록할 수 없다. 구체적으로는 `createTerms`, `uploadTermsList`, `registerTermsBatch` 모두 등록 전에 구성 단어의 `APRV_YN = 'Y'`를 체크하고, 하나라도 미승인이 있으면 "이 단어가 아직 승인되지 않았다"며 실패시킨다. 표준 관리의 의미상 맞는 설계였다. 승인 안 된 단어로 용어가 만들어지면 그 용어도 결국 승인이 안 되니까, 차라리 앞에서 막자는 것이었다.

둘러보니 UX가 이상했다. 일반 사용자가 표준화 추천에서 한글 컬럼명을 올리면 분석 결과에 미등록 단어가 포함되는 경우가 많다. 이 단어를 먼저 등록한 뒤 관리자 승인을 기다리고, 승인된 다음에 다시 추천 화면으로 돌아와서 용어 등록을 시도해야 한다. 승인을 기다리는 동안 사용자는 자기가 어디까지 했는지 기억해두고 있어야 하고, 중간에 다른 일로 주의가 빠지면 맥락을 다시 짜맞춰야 한다. 실제로는 "등록 시도 → 에러 → 단어 등록 → 승인 대기 → 돌아와서 다시 등록 시도"의 네다섯 단계가 한 번의 의도에 필요한 셈이었다.

그래서 설계를 뒤집었다. 용어 등록 자체는 미승인 단어가 섞여 있어도 허용한다. 대신 **용어 승인 시점**에 구성 단어가 전부 승인됐는지 체크한다. 미승인 단어가 있으면 그제야 승인이 막힌다. 사용자는 중간에 끊기지 않고 "등록까지" 한 번에 갈 수 있고, 관리자는 승인 화면에서 "이 용어는 아직 단어가 승인 안 됐다"를 한 번만 보면 된다. 단어 자체는 관리자가 먼저 승인하면 된다. 순서가 사용자 동선에서 관리자 동선으로 옮겨간 거다.

이 반전이 `putStdAprvStat`에 새 로직을 하나 더 얹었다. 용어 승인 시

`selectUnapprovedWordsByTermsId`를 호출해서 미승인 단어가 있으면 승인을 거부하고 목록을 반환한다. 단어 반려 시에는 반대 방향의 `selectUnapprovedTermsByWordId`를 호출해서 "이 단어를 쓰는 미승인 용

어가 이만큼 있다"를 경고한다. 반려가 연쇄적으로 영향을 주기 때문에 무심코 반려하면 여러 용어가 같이 막힌다는 걸 관리자가 알아야 했다.

변경 이력 저장 시점

원래는 `createWord` 같은 등록 API가 호출될 때 바로 `TB_CHANGE_HISTORY` 에 이력을 찍었다. 단어가 만들어지는 순간이니 기록을 남기는 게 자연스러워 보였다. 그런데 승인 프로세스가 붙으면서 이 시점이 어긋나기 시작했다. 등록만 하고 승인이 안 된 단어는 "역사에 남길 만한 사건"이 아닌데, 이력 테이블에는 들어가 있다. 반려된 경우까지 합치면 "실제로 표준이 된 적이 없는 것들의 이력"이 쌓였다.

규칙을 바꿨다. 이력은 **승인 완료 시점에만** 남긴다. 등록 API에서는 이력 저장 코드를 전부 제거하고, `putStdAprvStat` 이 승인을 처리할 때 이력을 INSERT한다. 반려는 이력을 남기지 않는다. 예외는 관리자가 직접 등록하는 경우다. 관리자는 자기 등록에 `APRV_YN='Y'` 가 즉시 붙으니 "등록=승인"이고, 이 경로에서만 등록 API에서 바로 이력을 저장한다.

코드 관점에서는 조금 번잡해졌다. 이력 저장 지점이 여러 곳에 흩어지는 대신 한 곳(승인 처리)에 모였지만, 관리자 경로는 예외로 남아있다. 이 예외를 지우려면 관리자 등록도 "자동 승인"으로 우회시켜서 승인 처리 API를 거치게 만들면 되는데, 거기까지는 가지 않았다. 관리자가 단어 하나 등록할 때마다 내부적으로 승인 워크플로우 API를 한 번 더 타는 건 오버라고 판단했다.

비관리자 테스트에서 드러난 권한 구멍

구현이 끝난 날(2026-04-01) 관리자 계정(admin)으로는 모든 시나리오가 통과했다. 그 다음 날 일반 사용자 계정(jyjang)으로 같은 시나리오를 돌렸더니 네 군데가 무너졌다.

첫째, 비관리자가 등록한 단어가 `APRV_YN='N'` 으로 들어가야 하는데 일부 경로에서 `APRV_YN` 필드 자체가 null로 들어갔다. `NULL != 'N'` 이고 `NULL != 'Y'` 이라 조회 쿼리의 `WHERE APRV_YN = 'Y'` 조건에서 걸러지지도 않고, 동시에 "승인된 것"으로도 취급되지 않는 회색 상태가 됐다. DB 스키마에 `NOT NULL + DEFAULT 'N'`을 걸어 쿼리 레벨에서 누락돼도 안전하게 떨어지도록 했다.

둘째, "내 요청 현황" 화면을 일반 사용자가 열었을 때 빈 목록이 나왔다. 쿼리는 맞았는데, `WHERE REG_USER = #{userId}` 바인딩에 관리자 세션 기준 `userId` 가 들어가고 있었다. 세션 컨텍스트에서 현재 사용자 ID를 꺼내는 헬퍼가 로그인 사용자가 아니라 API 토큰 소유자를 반환하는 구조였다. API 호출 주체와 "내가 만든 것"의 주체가 어긋나는 경우였다. 헬퍼를 두 갈래로 쪼갰다.

셋째, 승인 관리 화면에 비관리자가 접근할 수 있었다. 프론트 라우터에 역할 가드가 걸려 있었는데, 주소를 직접 치면 그 가드를 우회해서 화면이 렌더링됐다. 화면은 떴지만 API가 403을 던졌기 때문에 실제로 데이터는 못 봤다. 그래도 "승인 관리라는 메뉴가 있다는 것" 자체가 비관리자에게 노출된 건 문제였다. 가드를 API 레이어뿐만 아니라 메뉴 빌더 단계에서도 한 번 더 걸었다.

넷째, 반려 사유가 저장은 되는데 화면에 표시되지 않는 건이 있었다. 이건 필드명 오타였다. `rjctRsn` vs `rjtRsn`.

"내 요청 현황"이라는 별도 메뉴

승인 화면을 리디자인하면서 한 가지 구조적 결정을 같이 내렸다. 그동안 승인 화면은 관리자 전용이었다. 일반 사용자는 자기가 등록 요청한 게 어떻게 됐는지 볼 곳이 없었다. 관리자가 승인·반려를 해도 사용자는 그 결과를 확인할 수 없었고, 반려되면 사유도 몰랐다. 이게 "승인이 아래로 흐르지 않는" 불편함의 원인이었다.

두 가지 선택지가 있었다. (1) 기존 승인 화면에 역할별 필터를 추가해서 일반 사용자는 자기가 요청한 것만 보게 한다. (2) "내 요청 현황"이라는 별도 메뉴를 만든다.

(1)은 화면 하나로 정리되는 대신, 관리자 모드와 사용자 모드에서 같은 화면이 완전히 다르게 동작하게 된다. 버튼 종류도 다르고, 필터도 다르고, 일괄 승인 같은 기능은 관리자에게만 보여야 한다. 같은 컴포넌트를 역할에 따라 가지치기하는 조건문이 많아질수록 유지보수가 어려워진다. 그래서 (2)로 갔다. 메뉴를 "내 요청"이라는 대메뉴로 따로 만들고, 그 아래 "내 요청 현황"을 뒀다. 일반 사용자는 이쪽을 본다. 관리자는 여기서도 자기 요청을 볼 수 있지만, 실제 승인 처리는 "관리 > 승인 관리"에서 한다.

두 화면이 논리적으로 겹치는 부분은 있다. "내가 만든 요청"이라는 공통 개념이 있고, 차이는 "그 외에 누구의 요청을 볼 수 있는가"다. 그래도 화면을 분리한 덕에 각 화면의 상태 기계가 단순해졌다. 내 요청 현황은 조회 + 상세 보기, 승인 관리는 조회 + 상세 보기 + 승인/반려 + 일괄 처리. 두 번째 화면이 확실히 더 무겁다.

반려와 재요청, 그리고 상태 머신 확장

초기 설계에서는 반려 후에 사용자가 다시 제출할 수 있는 명시적인 경로가 없었다. 반려된 건은 그냥 "반려됨" 상태로 죽었고, 사용자는 새로 등록 요청을 올리는 방식이었다. 이게 불편한 이유는 두 가지였다. 하나, 반려된 원본과 새 요청 사이의 연결이 끊긴다. 관리자가 "이거 아까 내가 반려한 그건가?"를 추적할 수 없다. 둘, 반려 사유를 보고 고쳐서 재제출하는 흐름이 "새 등록"으로 취급되기 때문에 사용자 UX에서 "수정해서 다시 올린다"는 자연스러운 개념이 살지 않는다.

그래서 상태 머신에 재요청 전이를 추가했다. `REJECTED` 상태에서 사용자가 "수정해서 다시 제출"을 누르면 동일한 `OBJ_ID`를 유지한 채 `status`를 `REVIEW`로 되돌린다. 반려 사유는 이력에 남아 있고, 새 요청에서는 그 사유를 같이 보여준다. 관리자 입장에서는 "이전에 반려한 건이 다시 올라왔다"는 문맥을 바로 알 수 있다. 단어에 대해서는 2026-04-03 커밋에서 이 흐름이 들어갔고, 용어·도메인에도 같은 구조가 확장됐다.

그때 내린 타협

승인 프로세스의 많은 부분은 타협이다. 엄격한 설계(다형 참조 대신 네 테이블, 등록 시 차단, 이력을 등록 시점에 찍기)가 각 지점에서 선명한 대안이었지만, 운영 동선·사용자 동선을 고려해서 덜 엄격한 쪽으로 움직였다. 다형 참조의 FK 부재는 규칙으로 뒀었고, 등록 시 차단은 승인 시 체크로 뒤집었고, 이력 저장 시점은 관리자 예외를 안고 승인 완료로 옮겼다. 세 군데 전부 "이게 정답은 아닌 것 같은데, 지금 시점에선 이게 낫다"는 감각으로 결정한 것들이다.

이 감각이 맞았는지는 시간이 지나봐야 안다. 다형 참조가 언젠가 타입 누락으로 문제를 일으킬 수도 있고, 미승인 단어가 포함된 용어가 쌓여서 승인 화면이 경고 목록으로 도배될 수도 있다. 그럴 때 이 타협을 되돌릴지, 아니면 다른 방향으로 갈지는 그때 판단하면 된다. 지금은 이대로 돌린다.